AI Assistant

BTFViewer is an **AI-assistant tool for RTOS trace analysis**: find evidence and explain. This file is the technical and user-facing reference for the **AI** tab — investigation, model configuration, GUI tools, workflows, evaluation, and implementation.

For day-to-day panel usage and the product-level AI overview, see README.md → AI Assistant. For repeatable ask-order playbooks, see WORKFLOWS.md.

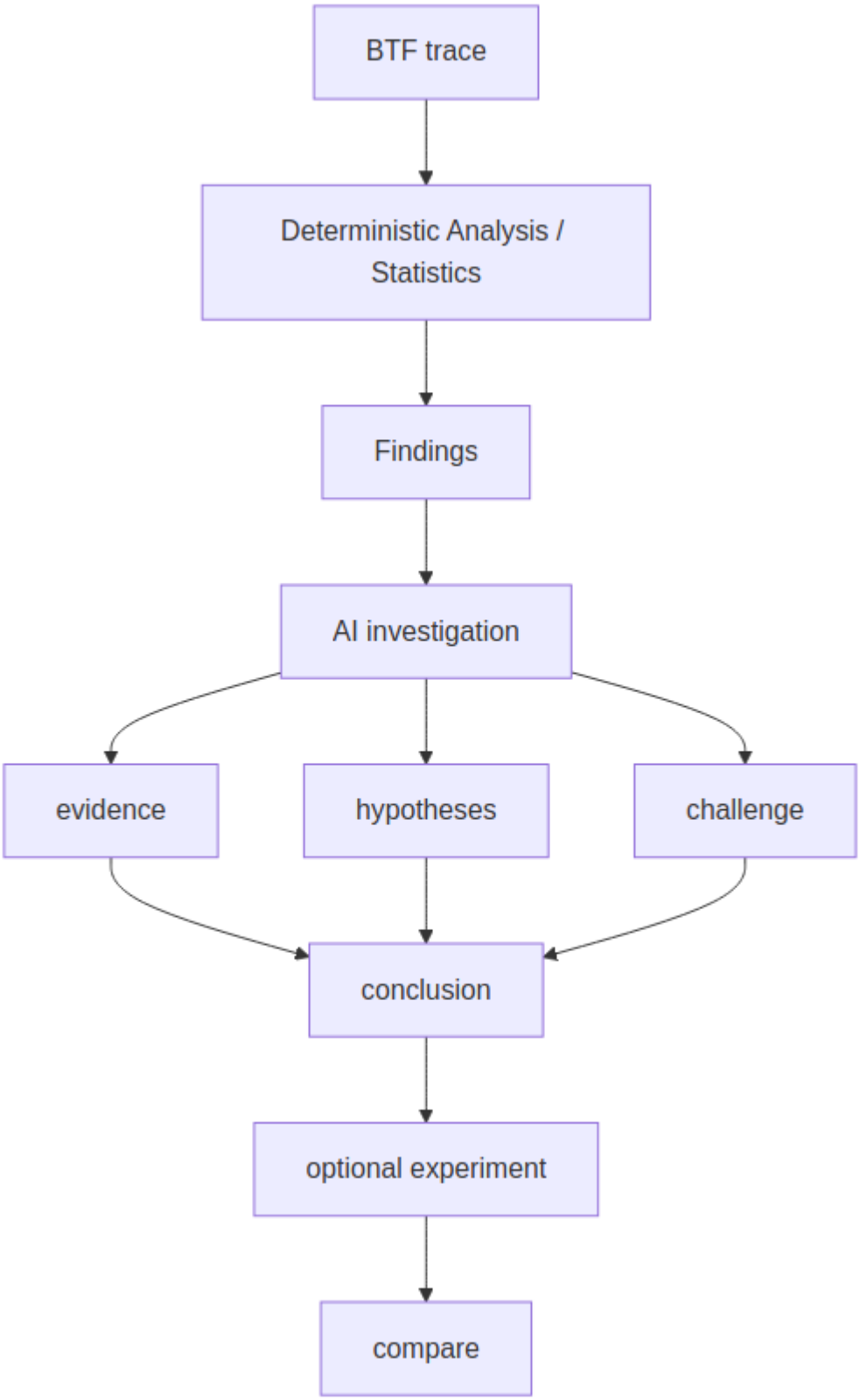
> **Core boundary:** the AI works over structured BTFViewer findings, statistics, timeline queries, and comparison results. It does not read firmware source/ELF files, and `what_if` is a labelled heuristic slice-replay estimate rather than a FreeRTOS kernel simulation.

Documentation map

`AI.md` is the **AI system reference**, not the primary step-by-step troubleshooting guide.

Document	Responsibility
README.md	Product and user-facing AI entry: find evidence and explain
STATISTICS.md	Deterministic measurements AI consumes
WORKFLOWS.md	Practical investigation sequences and ask order
AI.md	Models, tools, planner, evidence validation, evaluation, implementation

AI boundary


BTF trace
Deterministic Analysis / Statistics
Findings
AI investigation
evidence
hypotheses
challenge
conclusion
optional experiment
compare

AI may organize, explain, correlate, challenge, and estimate. It must not present heuristic estimates as measured trace results.

Documentation boundary

- README.md — user-facing AI entry point and product behavior
- WORKFLOWS.md — repeatable diagnosis and AI ask order
- STATISTICS.md — deterministic measurements consumed by AI
- **AI.md** — models, tools, planner, evidence validation, evaluation, and implementation

Contents

How it works

The **AI** tab answers diagnostic questions using structured **Analysis Findings** and summary metrics—not the raw `.btf` event stream. That keeps the prompt compact (token-efficient) and analysis fast. Toolbar **Compare** (right after **Analysis**, when two or more tabs are open) diffs the traces; the **Trace Compare** template / **Query with AI…** in that dialog sends those tables instead of Findings. Compare **Trends** lists every open tab; **Save as baseline** / **Score vs baseline** use the same stored profile as `baseline_score` (not a new tool). Analysis Findings **Save recipe…** / **Story…** are host chrome (user template / story export). **Ctrl+K** jumps to Analysis, AI, Compare, workspace presets, and Inspect task.

When a question needs granular per-task time-series, the model can call `query_raw_metric` to pull a scoped series on demand (still never the raw BTF file). `search_timeline` locates STI / tag / task timestamps like Find. `trigger_compare` returns Trace Compare CSV when two tabs are open. `detect_anomalies` ranks Findings as Critical / Warning / Info. `correlate_events` merges blocking / execution / migration / sync / priority events for one task. `find_critical_path` walks preempt/block/mutex around a timestamp. `compare_performance` returns structured A vs B deltas (two tabs). `regression_explain` narrates the primary A vs B change. `investigate` builds a root-cause chain plus hypotheses and suggested tools. `generate_report` returns typed engineering markdown (then `export_report` to save). `check_budget` compares WCET/response/deadline budgets; `optimize` returns qualitative mitigations; `what_if` runs a heuristic slice-replay simulator; `optimize_experiment` ranks automatic candidates; `analyze_traces` ranks all open tabs; `baseline_score` flags drift vs a stored baseline; `recommend_experiments` suggests bench follow-ups; `detect_priority_inversion` / `find_related_findings` / `compare_tasks` cover PI suspects, adjacent findings, and two-task deltas; `explain_finding` / `interpret_query` / `validate_experiment` / `manage_hypotheses` cover levelled explanations, query interpretation, experiment close-out, and hypothesis status; `bookmark_finding` pins semantic investigation marks; `investigation_replay` summarises a completed investigation. Query-only batches (the read-only tools in GUI tools) run immediately; mixed GUI batches wait

for **Apply** unless **Auto-apply GUI actions** is on. Export tools (`export_report`, `export_investigation`) still show a save dialog.

Important conclusions should cite evidence (`jump:TIME`, metric names) and state **confidence** (High / Medium / Low) plus evidence quality (Directly observed / Strong correlation / Possible explanation / Insufficient evidence). The system prompt asks the model not to invent numbers, task names, or `jump:TIME` values absent from findings, tool results, or Trace Compare tables — and, when a **Cursor region window** is listed, to cite only times inside that window.

The AI panel stepper (Triage → Scope → Investigate → Verify → Experiment → Compare) follows Findings and tools already run; click a completed stage to jump to that output in the log (the matching message flashes). Empty log: **Start Investigation** runs **Auto investigate**. **Clear** removes chat replies, resets the usage meter, and clears current investigation issues (the stepper returns toward idle / **Start Investigation**). The investigation case, plan, and recent chat restore after app restart **when the log still has a user or assistant turn** (desktop `[ai] investigation_session`; web session snapshot). An empty log (or Clear then quit) does not restore a Current Issue card — **Start Investigation** stays available. What-if / Optimize stay on **Verify** until verify tools or strong evidence quality; the Experiment banner is a heuristic estimate (recapture a trace and Compare to measure). **Investigate / Root cause / Verify finding / Auto investigate / What-if / Optimize / Diagnostic report** show an **Investigation plan** checklist (steps advance as tools run; the final text reply marks remaining steps done). Analysis Findings may include anomaly rows (WCET Max≫Avg spikes, extreme migration bursts). **Explain finding** (Analysis **Explain…**: Quick / Technical / Deep) calls `explain_finding` with `level=`. Mode chips and **More → Investigations** (including **Save as template…**) start tool sequences without adding templates after Auto investigate. Diagnose / Investigate / Auto investigate walk graph → temporal → `rank_root_causes` → `challenge_conclusion` before `what_if`. Evidence hypothesis rows offer Support / Reject / Need evidence / Test / Compare.

Right-click a timeline segment → **Ask AI about this event** to ask about that exact task/segment (`Explain the timeline event for task … around jump:TIME`), same as Explain region but scoped to one segment. With **≥2 cursors**, the timeline context menu also offers **Explain this region with AI** (`explain_region`); the AI panel **Explain region** template is always available — without two cursors it runs on full-trace Findings (no region window). When **Settings → AI** is off, those timeline items are grayed (they open Settings if clicked). When cursors are placed, the prompt includes an explicit **Cursor region window** (`jump:lo … jump:hi`) so replies

should stay in-window. When `investigate` returns a root-cause chain and hypotheses, the Evidence panel renders a small **Investigation tree** plus an **Evidence Quality** meter (Strong / Medium-High / Medium / Weak — a diagnostic heuristic, **not** a probability). Direct evidence times, timeline correlation, and metric checks raise the band; untested alternatives lower it. The panel also lists **what would disprove** the conclusion, **evidence coverage**, and an **evidence graph** (finding → evidence → hypotheses). After the final reply, a host-side **validator** flags invented task names and `jump:TIME` values outside the cursor window.

Natural-language timeline questions (e.g. “find STI wait around TaskA”) are answered via `search_timeline` — no separate search UI.

Recommended ask order (WORKFLOWS.md §7): triage overall findings → **Investigate** / **Root cause** when you want tool-driven drill-down → named metric templates → mitigations after the timeline agrees. Prefer the built-in templates; they already name the metrics and units, and they point at Statistics pages that already exist (**Timeline Anomalies**, **Worst Events**, **Response Time**, **Critical Path**, **Period / Jitter**, **Unified Jitter**, **Recurring Patterns**, **Task Health**, **Task × Core**, **Core Utilization Over Time**, **Preemption Matrix**, **Waiter × Owner**, **Mutex Blocking**) — there is no extra `detect_timeline_anomalies` tool. Concrete flows and use cases are below.

Workflows and use cases

For the practical symptom-to-metric ladder and repeatable investigation sequences, use **WORKFLOWS.md**. This section documents the AI-side behavior and capabilities behind those workflows.

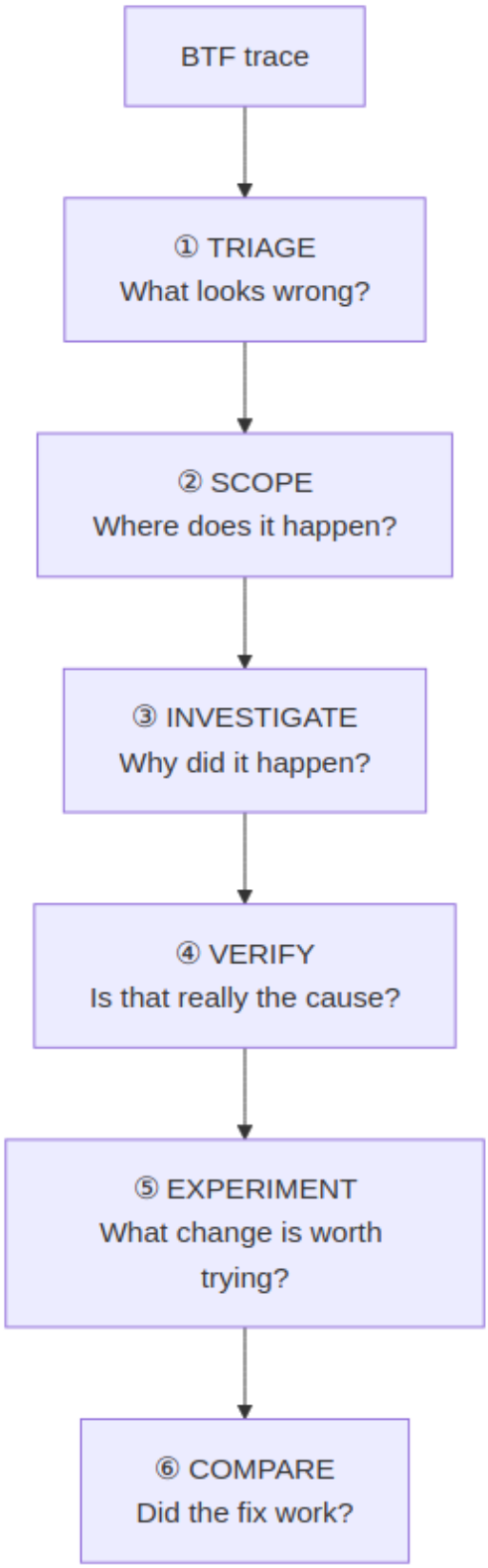
Same flow on **Desktop** and **Web**. Panel chrome: README → AI Assistant. Ladder and ask-order tables: WORKFLOWS.md §7.

In this section

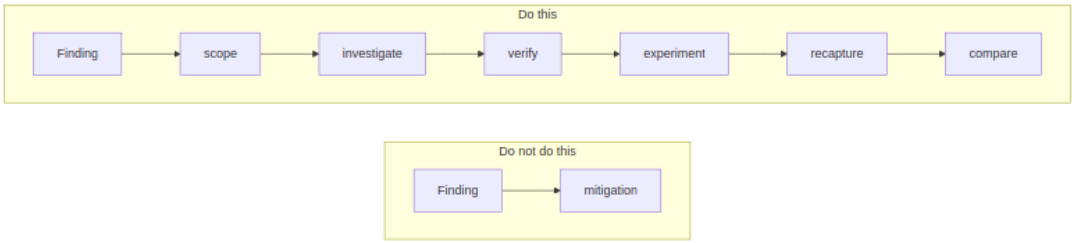
In this section

Newbie AI workflow — "I don't know what is wrong"

For a first investigation, do **not** start by choosing individual tools. Use the six-step loop below and let **Investigate** choose the deeper tools.


BTF trace
① TRIAGE
What looks wrong?
② SCOPE
Where does it happen?
③ INVESTIGATE
Why did it happen?
④ VERIFY
Is that really the cause?
⑤ EXPERIMENT
What change is worth trying?
⑥ COMPARE
Did the fix work?

Step	Start with	What you should get	What you must verify
1. Triage	**Triage findings** or toolbar **Analysis**	Ranked Critical / Warning / Info issues and a suggested place to start	The named Statistics page contains the same problem
2. Scope	Select the finding; apply suggested cursors or place C1–Cn	One task / incident / time window instead of the whole trace	**Limit to C1–Cn** is on when the question is phase-specific
3. Investigate	**Investigate**; use **Root cause** when a suspect task is already known	Hypotheses, related evidence, critical path / dependency / temporal relationships	Click `jump:TIME`, `range:LO/HI`, and named Statistics sections
4. Verify	**Verify with AI…** or continue the Investigation plan	Supported / rejected / insufficient conclusion, alternatives, contradictions, evidence quality	Evidence is in scope; task names and times are real; alternatives were challenged
5. Experiment	**What-if**, **Optimize**, or experiment plan	Ranked *estimated* pin / priority / contention / migration changes	Treat the result as an estimate only; change firmware/configuration and recapture
6. Compare	Open before/after tabs → **Compare** → **Query with AI…** / **Validate experiment…**	Measured deltas and whether the expected improvement happened	Same workload phase and comparable cursor scope on both traces



The beginner rule

A useful first session therefore needs only these user-facing actions:

If you want to…	Use
Find the biggest issue	**Triage findings**
Understand one issue	**Investigate**
Check whether the explanation is real	**Verify with AI…**
Understand one selected time range	**Explain region**
Test a possible change	**What-if / Optimize**
Prove the change helped	**Trace Compare**

Individual tool names such as `correlate_events`, `find_critical_path`, or `rank_root_causes` are mainly useful for advanced users and developers. The normal workflow should be driven by the templates and Investigation plan.

End-to-end flow

Do not ask for mitigations before the timeline agrees with the finding. Empty or mis-scoped Statistics produce confident nonsense. Prefer built-in templates: they already name metrics and units.

Investigation workflow

Step	Template or tool	Why
1	**Triage findings** / `detect_anomalies`	Rank Critical / Warning / Info; open Timeline Anomalies / Worst Events / Task Health
2	**Investigate** / `investigate` — or Findings **Auto investigate…**	Root-cause chain, hypotheses, alternatives, suggested tools

Step	Template or tool	Why
3	**Verify finding** / Findings **Verify with AI…**	Confirmed / Rejected / Inconclusive with jump:TIME evidence
4	`correlate_events` + `query_raw_metric`	Merge blocking / execution / migrations / sync / PI for one task
5	`find_critical_path` / `detect_priority_inversion`	Preempt/block path; L/M/H inversion suspects
6	`build_task_dependency_graph` / `analyze_temporal_causality`	BTF wait/preempt/migrate chain
7	`rank_root_causes` / `challenge_conclusion`	Rank then alternatives before `what_if`
8	`find_related_findings` / `compare_tasks`	Adjacent findings; side-by-side task deltas
9	`set_cursors` / `zoom_to_range` / `highlight_task` / `bookmark_finding`	Narrow the timeline (Apply cursors unless auto-apply is on); click `range:L0/HI` / `btfrange:` on critical-path evidence
10	Evidence panel	Investigation tree + **Evidence Quality** + what would disprove this
11	`investigation_replay` / `generate_report` / `export_investigation`	Structured close-out; optional `export_report`

Root cause walks deadline/WCET → preemption → blocking → mutex → inheritance → migration for the top finding. Use it when triage already named a suspect task.

Auto investigate chains verify-style steps for one finding (investigate → correlate → critical path / graph / temporal → rank → challenge → what-if) and advances the Investigation plan checklist. Use **Verify** when you already have a finding id and want a short verdict.

Explain region and Ask event

Entry	When it appears	Scope
Timeline → **Explain this region with AI**	Only with **≥2 cursors**; grayed when AI is off	C1–Cn; prompt gets `Cursor region window: jump:lo … jump:hi`
AI panel → **Explain region**	Always	Same window when ≥2 cursors; **full-trace Findings** if none
Timeline segment → **Ask AI about this event**	Segment under the pointer; grayed when AI is off	One task / core / segment around `jump:TIME`

Stay inside the stated window: every `jump:TIME` in the reply should fall between C1 and Cn (or the model should say the window has no matching evidence). Enable **Limit to C1–Cn** so Statistics / Findings / `query_raw_metric` match that window. Clicking a `jump:TIME` outside the cursors usually means the model invented or reused a full-trace time — discard it and re-ask with cursors + scoped Findings.

What-if and optimize workflow

`what_if` and `optimize_experiment` are **heuristic slice-replay** tools: they reallocate measured execution slices, scale migrations / blocking, and adjust core-util balance. They are **not** a FreeRTOS kernel or deterministic scheduler. Every result carries a disclaimer. After a promising estimate, `recommend_experiments` suggests validation steps (simulation / firmware / measurement).

Goal	What to run	Typical change phrases
One concrete idea	**What-if** → `what_if`	`pin CS[28] to Core_0`, `raise priority of Low[266]`, `reduce mutex contention 50%`
Rank several ideas	**Optimize** → `optimize_experiment` (then optional `optimize` for qualitative notes)	Host picks pin-to-dominant / quiet core, contention −50%, priority up, migrations −50%
Soft advice only	`optimize`	Finding-text mitigations without scored experiments
What to try next on the bench	`recommend_experiments`	Validation experiments from findings heuristics

Reading the payload: compare `baseline` vs `simulated` (migrations, blocking_ns, load_balance_score) and the `deltas.cost` ranking. Lower cost is better in the experiment list. Treat Medium confidence as “worth trying on the bench”; Low means the phrase was vague or slices were thin.

Use cases

Situation	Before you ask	Template / tools	Then verify
Unknown — first look	Full-trace or cursor-scoped Findings	**Triage findings** → **Investigate**	Timeline Anomalies / Worst Events / Task Health; `jump:TIME`

Situation	Before you ask	Template / tools	Then verify
Hottest / noisiest task	Findings name a suspect	**Task profile**	Period / Jitter; Task Health; Task × Core; Execution / Blocking p95/p99
Tick jitter / missed ticks	Trace Health in scope	**Tick health**	Tick Distribution (not Period / Jitter — that page is task inter-arrival)
Confirm one finding	Select it in Analysis Findings	**Verify with AI…** / **Verify finding**	Evidence panel; timeline
Explain a time window	≥2 cursors; **Limit to C1–Cn** on	Context menu or **Explain region**	Only `jump:TIME` inside C1–Cn
One segment / ISR slice	Right-click the segment	**Ask AI about this event**	That task's row; nearby STI
Auto walk a finding	Select finding → **Auto investigate…**	`auto_investigate`	Investigation plan + Evidence
Migration thrash / ping-pong	Scope thrash window; Findings mention the task	**Migration thrash** → `correlate_events` → **What-if** pin / **Optimize**	Task × Core; Timeline Anomalies migration bursts; Migrations Rate/Ping; Heatmap / Chord; Core Affinity

Situation	Before you ask	Template / tools	Then verify
Priority inversion / PI boost	Inheritance finding or PI episodes in scope	**Priority inversion** / `detect_priority_inversion` → `find_critical_path`	Priority Inheritance; Mutex hold; Waiter × Owner (heuristic handoff)
High blocking / mutex wait	Scope the stall; suspect task known	**Highest latency** → `query_raw_metric` blocking/sync → **What-if** contention / priority	Worst Events; Waiter × Owner; Blocking p95/p99; Mutex hold; Priority Inheritance
WCET / deadline pressure	Thresholds set in Display → Analysis	**WCET / hot CPU** or **Deadline / budget** → `check_budget`	Timeline Anomalies / Worst Events; Period / Jitter; Task Health; Execution Max / p95 / p99; Deadlines
Compare two tasks	Both names known	`compare_tasks`	Execution / Blocking / Migrations side-by-side
Related findings	One finding selected	`find_related_findings`	Shared task / metric / nearby times

Situation	Before you ask	Template / tools	Then verify
Load imbalance across cores	Multi-core util in Findings	**Core balance** → `analyze_traces` (multi-tab) or **What-if** pin to quiet core	Task × Core; Load Balance Score; Concurrent Active; Core Time Breakdown
A vs B build regression	Two tabs open	**Trace Compare** (toolbar **Compare** → **Query with AI…**) / `compare_performance` / `regression_explain`	Compare summary strip; Trace Compare pages; same scope on both builds
Drift vs saved baseline	Baseline profile stored (rc / localStorage)	`baseline_score`	Flags `|z|>2`; re-capture if needed
Rank all open traces	≥2 loaded tabs	`analyze_traces`	Best tab vs Migrations / LB / missed ticks
Write-up for a review	Cause already confirmed	**Diagnostic report** → `generate_report` → `export_report` / `export_investigation`	Saved HTML/CSV/JSON; evidence times bookmarked

Situation	Before you ask	Template / tools	Then verify
CI gate vs baseline	Desktop CLI	`[analyze](#cli-regression-gate)` `--fail-on-regression` (optional `--ai`)	Exit code + markdown narrative

Worked examples

Migration thrash → pin affinity

1. Place cursors on the thrash window; enable **Limit to C1–Cn**; re-open Analysis.
2. Run **Migration thrash** or **Investigate** until the hot task (e.g. `CS[22]`) and cores match the heatmap.
3. Ask **What-if**: *pin CS[22] to its dominant core* (or run **Optimize** for ranked candidates).
4. Read Δmigrations / Δload_balance_score. If migrations drop but LB worsens sharply, try pin-to-quietest via `optimize_experiment` and compare ranks.
5. On firmware: set affinity / reduce bounce; re-capture a `.btf` and toolbar **Compare** the before/after tabs.

Mutex contention → shorter critical section

1. **Highest latency** / `correlate_events` for the waiter; confirm hold episodes in Mutex / Priority Inheritance.
2. **What-if**: *reduce mutex contention 50% for TASK* (or another %).
3. Expect lower `blocking_ns` in the simulated payload — still an estimate. Confirm by shortening the hold in code and re-tracing.

Two builds → regression narrative

1. Open baseline and candidate as tabs; match cursor scope if needed.
2. Toolbar **Compare**, then **Query with AI…** (**Trace Compare** template), or `compare_performance` then `regression_explain`.
3. Act only on High/Medium confidence deltas that Statistics on both tabs reproduce.
4. Optional: `optimize_experiment` on the candidate's hottest task to sketch mitigations (still heuristic).

Cursor window → Explain region

1. Place **C1** / **C2** on the phase of interest (e.g. 1.060 s … 1.120 s); enable **Limit to C1–Cn**; re-open Analysis.
2. Right-click the timeline → **Explain this region with AI** (or AI panel → **Explain region**).
3. Confirm the user turn lists `Cursor region window: jump:lo … jump:hi`. Reject any `jump:TIME` outside that window.
4. Follow up with `correlate_events` / `query_raw_metric` on tasks named in-window; bookmark evidence times.

Priority inversion finding → Verify

1. Open Analysis Findings; select a Priority Inheritance / inversion row.
2. **Verify with AI…** (or **Auto investigate…** for a longer chain).
3. Expect `detect_priority_inversion` / `query_raw_metric` (priority_inheritance) / `find_critical_path`; read the Evidence score and investigation tree.
4. Click in-scope `jump:TIME` links; confirm L/M/H on the timeline and in Priority Inheritance statistics.

Simulator limits

Does	Does not
Replay measured slices / migrations / blocking gaps for the Statistics scope	Run FreeRTOS scheduling, ISRs, or cache models
Score pin / priority / contention / migration experiments	Guarantee WCET or deadline after a firmware change
Label every result as estimate / not measured	Replace timeline verification or a new capture

Phrase changes as **pin** / **affinity** / **priority** / **mutex** / **migration** so the simulator engages; vague text falls back to a qualitative estimate (`simulator: none`).

Endpoints and models

Any OpenAI-compatible endpoint works, including Ollama (`http://localhost:11434/v1`). Chat requests time out after 120s (**Stop** still cancels sooner). Give the endpoint at least an **8k** context window so a full Findings card plus a tool round still fits.

Local models: the shipped Ollama default is `qwen3.5:9b`. Pull with `ollama pull qwen3.5:9b`. Larger local ids (`qwen3.5:27b`, `qwen3.8:27b`, `gemma4:26b`) trade latency for capacity. 3B-class models often skip native tools, dump tool JSON as text, and fail the investigation suite — do not use them.

Import presets: `[examples/ai](examples/ai/README.md)` ships `[ollama.json](examples/a` `[gemini.json](examples/ai/gemini.json)`, `[openai.json](examples/ai/openai.json)`, `[deepseek.json](examples/ai/deepseek.json)`, `[grok.json](examples/ai/grok.json)`, and `[presets.json](examples/ai/presets.json)`. Unknown preset names are added to the combo. Imported values fill **Settings → AI** (including checkbox flags when the file names them); review and confirm to save. Each preset keeps its own base URL, model, key, auth mode, and TLS flag.

Keys follow the same rules on Desktop and Web: Settings → AI → API key first, then `OPENAI_API_KEY`, then `GEMINI_API_KEY`, then `OLLAMA_API_KEY`. A local Ollama endpoint needs no key. Custom agents (Cursor, …) have no extra env name in the GUI — paste the key on that preset. Live `ai-test` XML may use `<api-key env="VAR">`. Full examples: README → API keys.

AI capability / model matrix

Capability	Small local	Local 9B+	Cloud
Basic Q&A	✓	✓	✓
Tool calling	△	✓	✓
Investigation (`investigate` / root-cause chain / hypotheses)	△	✓	✓
Complex reasoning (multi-step correlation, alternatives)	△	✓	✓
Large traces (big Findings card / long chat history)	△	△	✓
What-if / optimize (`what_if`, `optimize_experiment`)	✓	✓	✓

✓ reliable · △ inconsistent — works sometimes but often skips native tool calls, hallucinates numbers, or truncates on long context; always verify against the timeline before trusting a result.

Which model should I use?

If you…	Use
Want a local investigator (no key)	`qwen3.5:9b` — shipped Ollama default; best practical local on the investigation suite (~52s/case, 78 overall)

If you…	Use
Need more local quality	`qwen3.8:27b` (88, ~190s/case) or `qwen3.5:27b` (81, ~149s/case). `gemma4:26b` is slower than 9b and slightly worse (73, ~111s/case)
Have a large scope (many findings, long chat) or want the strongest reasoning	Cloud (`gpt-4o`, Gemini, DeepSeek, Grok) — mind the privacy trade-off
Handle confidential traces	Local Ollama regardless of size — nothing leaves the machine

Small local models often skip native tool calls and emit a fenced “‘`btftool` block instead — the viewer renders the same GUI cards either way (see GUI tools) — but investigation-heavy templates (Investigate / Root cause / Verify / Explain region / Auto investigate / Ask AI about this event / What-if / Optimize) need a tool-capable model such as `qwen3.5:9b` to reliably chain multiple calls.

Credential storage

	Desktop	Web
Where keys live	`[ai] *_api_key` in `btf_viewer.rc` next to the viewer	Browser `localStorage` (`btf-viewer-settings-v1`)
At rest	Encrypted as `enc1:…` (machine-bound; not portable to another host)	**Plaintext** in localStorage — treat as convenience only
Sent to the model	Never as a chat field; only as the HTTP `Authorization` / API header to the configured endpoint	Same
Clear	Settings → AI → clear key, or delete `btf_viewer.rc` AI keys	Settings → Reset / clear site data

What leaves the machine

```
What AI receives
✓ Analysis Findings (titles, severities, task names, heuristic text)
✓ Selected metrics / tool results the model requests
✓ Scoped timeline search hits and correlate windows
✓ Trace Compare CSV when that template runs
✓ User question + short chat history

What AI does NOT receive
✗ Raw .btf / .btf.gz file bytes
✗ The full unrequested event stream
✗ API keys inside the prompt body
```

	Local Ollama	Cloud endpoint
Trace file stays local	✓	✓
Findings / metrics leave the machine	No (loopback)	Yes — to that vendor
Raw BTF uploaded	No	No
API key required	Usually no	Usually yes

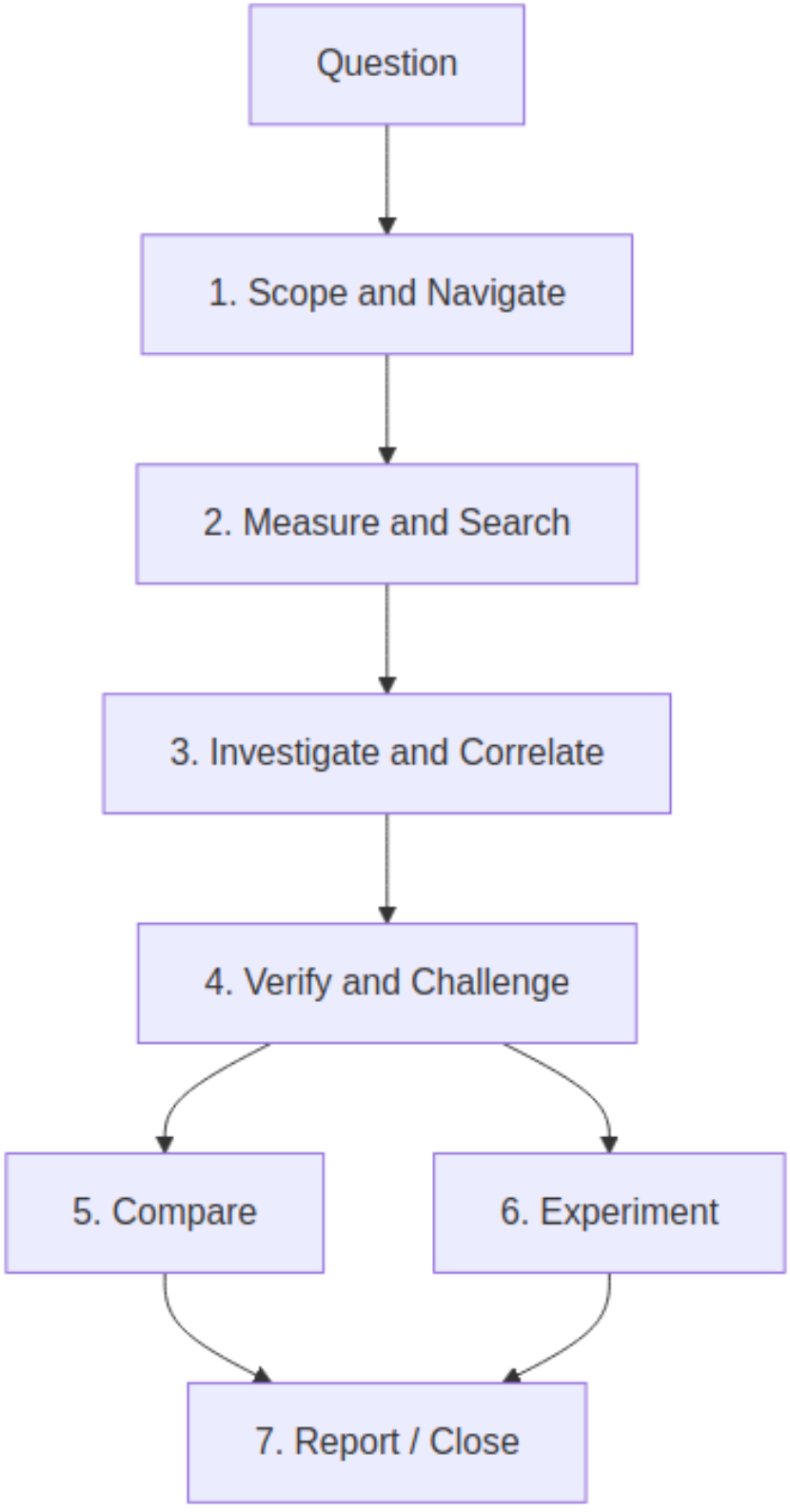
Prefer local Ollama for confidential traces. Redact sensitive task names in annotations before using a cloud preset.

GUI tools

Read-only tools are listed in the complete reference below.

The AI tool set is easier to understand by **purpose** than by function name. A normal user should start with the built-in templates and Investigation plan; the individual tool schema is primarily an advanced/debugging reference.

Tool mental model



Apply, Skip, and Undo

Tools fall into two behavioral classes:

Class	Behavior	Examples
Read-only evidence tools	Run immediately; they do not change the viewer	`query_raw_metric`, `search_timeline`, `investigate`, `correlate_events`, `find_critical_path`, `verify_claim`
GUI-changing tools	With **Auto-apply GUI actions** off (default), the batch waits for **Apply** / **Skip**	`set_cursors`, `zoom_to_range`, `highlight_task`, `set_view_mode`, `add_annotation`, `bookmark_finding`

Several tool calls may arrive in one model turn and are applied as one batch. **Undo last actions** restores zoom / view / highlight / inspector / marks; `Ctrl/Cmd+Z` also reverts cursors and marks. Export tools still open a save dialog.

1. Scope & navigate — "Where should I look?"

Use these tools to turn an answer into a visible timeline location.

Goal	Main tools	Result
Scope a suspected phase	`set_cursors`, `zoom_to_range`	Places C1–Cn and focuses the relevant interval
Focus a task	`highlight_task`	Lock-highlights the task on the timeline
Change perspective	`set_view_mode`	Task/Core and horizontal/vertical view
Inspect a migration corridor	`open_corridor_inspector`	Opens the Migration & Corridor Inspector

Goal	Main tools	Result
Preserve evidence	`bookmark_finding`, `add_annotation`	Adds semantic or free-text timeline marks
Clean up	`clear_marks`, `reset_view`	Clears investigation clutter or restores full-span view

Beginner usage: normally accept these as GUI cards produced by **Investigate**, **Verify**, or **Explain region** rather than calling them directly.

2. Measure & search — "What happened?"

These tools fetch deterministic evidence. They do not change the trace.

Question	Main tools	Evidence returned
Where did an event occur?	`search_timeline`	Matching task/STI/tag/interval/pointer/migration timestamps
What are this task's measured values?	`query_raw_metric`	Scoped execution, blocking, migration, sync, PI, or findings rows
Is the distribution unusual?	`analyze_distribution`	p50–p99.9, standard deviation, CV, outlier rate
Is timing periodic or jittery?	`analyze_periodicity`	Expected vs observed period/jitter statistics
Does the task exceed a budget?	`check_budget`	WCET/response/deadline budget comparison

These tools are the evidence layer. They should be preferred over asking the model to guess a number.

3. Investigate & correlate — "What is related?"

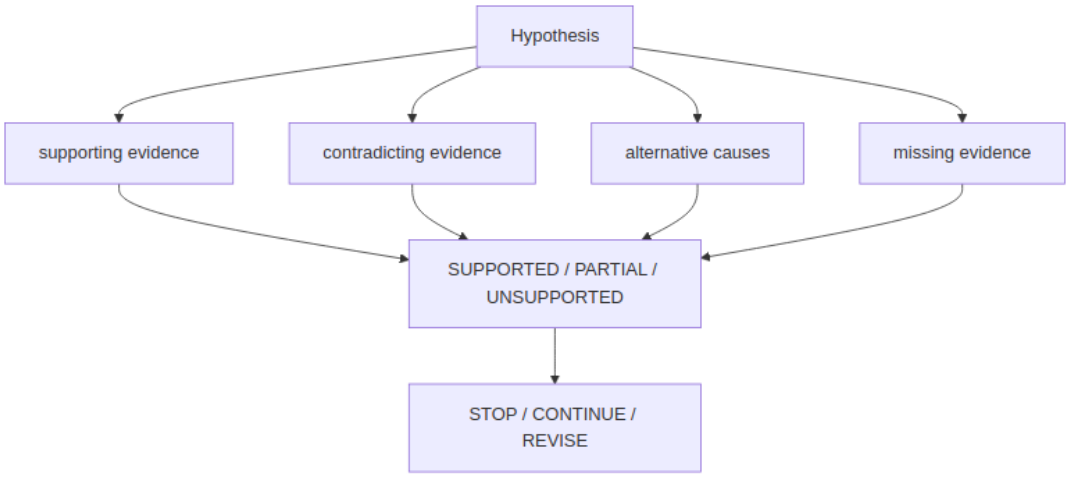
Use this group after Triage identifies a concrete issue.

Depth	Main tools	Purpose
Triage	`detect_anomalies`, `cluster_findings`, `cluster_incidents`	Rank and group Findings
Investigate	`investigate`, `plan_investigation`, `suggest_scope`	Build hypotheses and choose the cheapest next checks
Correlate	`correlate_events`, `find_related_findings`	Merge nearby execution/blocking/migration/sync/priority evidence
Path	`find_critical_path`	Walk preemption, blocking, and mutex activity around an incident
Dependency	`build_task_dependency_graph`	Show wait/preempt/migrate/PI relationships
Temporal	`analyze_temporal_causality`	Build a happens-before chain from observed evidence times
Root cause	`build_causal_chain`, `rank_root_causes`	Rank hypotheses while labeling causal/correlated/temporal relationships

`Investigate` / `Root cause` / `Auto investigate` orchestrate these deeper tools. Most users should not need to choose the sequence manually.

4. Verify & challenge — "Is this really the cause?"

This group prevents a plausible story from being treated as a confirmed diagnosis.



Purpose	Main tools
Check one claim	`verify_claim`
Find evidence against a hypothesis	`detect_contradictions`
Decide whether enough evidence has been gathered	`assess_evidence_sufficiency`
Force alternative explanations	`challenge_conclusion`
Track hypothesis state	`manage_hypotheses`
Inspect priority-inversion evidence	`detect_priority_inversion`
Evaluate investigation quality	`score_investigation`

The Evidence panel complements these tools with evidence quality, coverage, what would disprove the conclusion, and the investigation tree.

5. Compare — "What changed?"

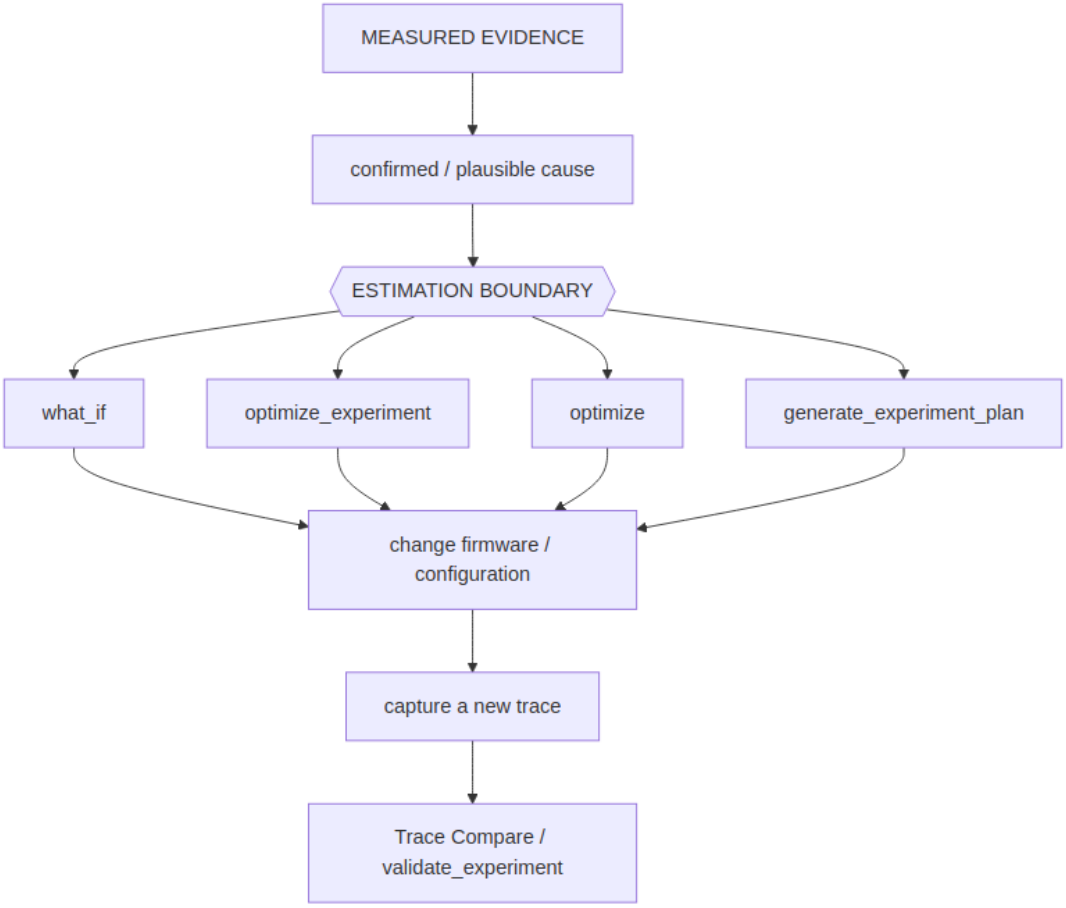
Use these tools when two or more traces are open.

Goal	Main tools	Result
Open / obtain Trace Compare	`trigger_compare`	Same comparison data as toolbar **Compare**
Compare two builds	`compare_performance`	Structured metric deltas + confidence
Explain the primary regression	`regression_explain`	Narrative tied to A/B deltas
Localize the regression	`regression_localize`	Suspect task / region / mechanism
Compare two tasks	`compare_tasks`	Side-by-side task metrics
Compare against history	`baseline_score`	Drift against a stored baseline
Rank several open traces	`analyze_traces`	Relative scheduling behavior

A comparison is only meaningful when the two traces represent equivalent workload phases.

6. Experiment — "What should I try?"

These tools operate **after** a cause has enough evidence.



Goal	Main tools
Test one concrete idea	`what_if`
Rank candidate changes	`optimize_experiment`
Get qualitative mitigation ideas	`optimize`
Generate bench/firmware validation steps	`recommend_experiments`, `generate_experiment_plan`
Compare prediction with measured result	`validate_experiment`
Store the outcome	`record_experiment_outcome`

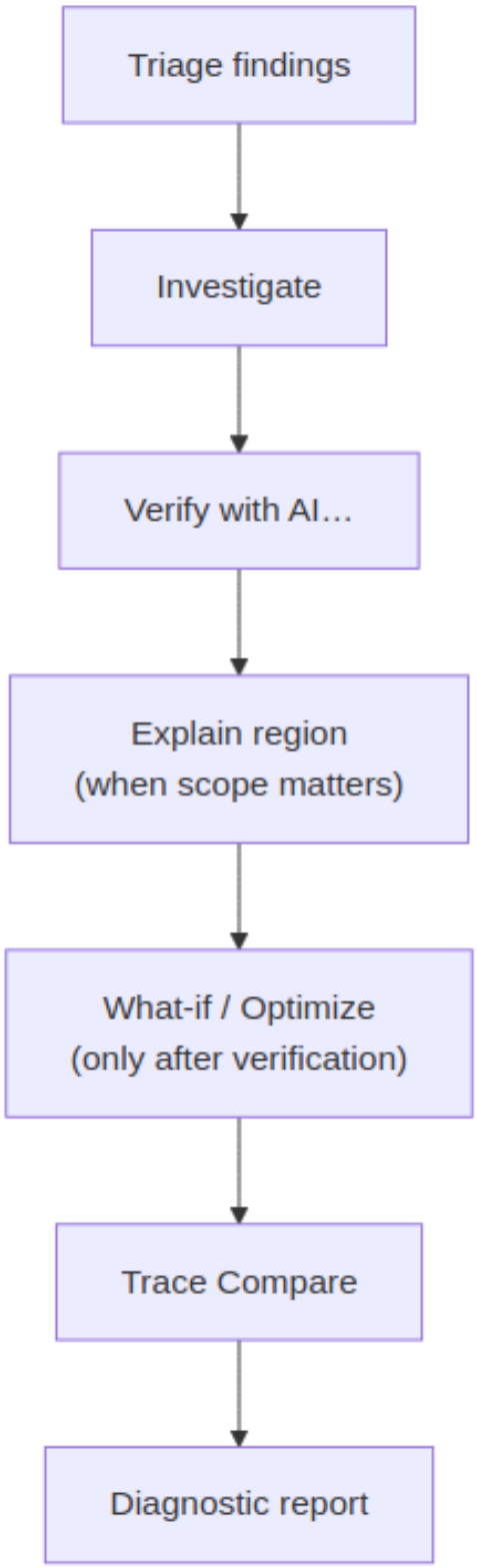
Important: `what_if` and optimization results are estimates, not measured scheduler behavior.

7. Report & close — "What did we learn?"

Goal	Main tools
Generate structured engineering text	`generate_report`
Save a report	`export_report`
Save a complete Investigation Case	`export_investigation`
Replay/summarize the investigation	`investigation_replay`, `summarize_investigation_context`
Remember similar cases	`investigation_memory`, `find_similar_investigations`
Finish the Case	`close_investigation`

Which tools should a beginner know?

Only a small subset needs to be visible in the mental model:


Triage findings
Investigate
Verify with AI…
Explain region
(when scope matters)
What-if / Optimize
(only after verification)
Trace Compare
Diagnostic report

Everything else is supporting machinery selected by the planner or used by advanced users.

Complete GUI tool reference

The table below is the exhaustive schema reference. Use it when implementing, debugging, or explicitly steering tool calls.

Tool	Parameters / targets	Effect
`set_cursors`	`timestamps` (1–8 trace times)	Place cursors (enables **Limit to C1–Cn** when two or more)
`zoom_to_range`	`start_time`, `end_time`	Focus the timeline between two times
`highlight_task`	`task_name_or_id` (display name, numeric id, or merge key)	Lock-highlight a task row. Unknown names are ignored so the timeline is not dimmed. Empty string clears.
`set_view_mode`	`mode` (`task` / `core`); optional `orientation`	Switch Task or Core view; horizontal or vertical
`open_corridor_inspector`	optional `core_from` / `core_to` (`Core_0`, `0`, `c0`, `Core 0`)	Open Migration Inspector; aliases resolve the same way
`add_annotation`	`time`, `note` (≤240 chars)	Pin an orange timeline note at a timestamp (stays on the current right-panel tab)

Tool	Parameters / targets	Effect
`query_raw_metric`	`task`, `metric` (`priority_inheritance`, `execution`, `migrations`, `blocking`, `sync`, `findings`)	Read-only: return the per-task series for the current Statistics scope (up to 40 rows)
`export_report`	optional `format` (`html` / `csv` / `json`)	Download HTML/CSV/JSON bundling Analysis Findings, mermaid diagrams from the chat, annotations, and GUI state (cursors / highlight / view); `json` saves a full investigation package (see `export_investigation`).
`clear_marks`	optional `what` (`annotations` / `cursors` / `bookmarks` / `all` / `everything`)	Clear AI clutter. `all` (default) drops annotations + cursors; `everything` also clears bookmarks
`reset_view`	(none)	Fit the timeline to the full span and clear the task highlight (marks stay)
`search_timeline`	`query`; optional `mode` (`contains` / `exact` / `regex` / `sti` / `tags` / `intervals` / `lifecycle` / `pointers` / `migrations`)	Find-panel search; returns matching timestamps (up to 40)

Tool	Parameters / targets	Effect
`trigger_compare`	optional `tab_a` / `tab_b` (0-based tab index or filename)	Read-only Trace Compare CSV + open the same dialog as toolbar **Compare** (needs two loaded tabs)
`investigate`	optional `finding_id`, `depth` (1–5)	Read-only: investigation graph with root-cause chain, hypotheses, ranked anomalies, suggested tools
`detect_anomalies`	optional `limit` (1–40)	Read-only: rank Analysis Findings as Critical / Warning / Info
`correlate_events`	`task`; optional `around_time`, `window`	Read-only: merge blocking / execution / migration / sync / priority / Find hits into one timeline

Tool	Parameters / targets	Effect
`find_critical_path`	`task`; optional `timestamp`, `window` (default 2000)	Read-only: preempt/block/mutex critical path around a timestamp; also returns a `mermaid` graph (`graph LR`), `graph_nodes` (id/label/kind/time), and split `blocking_steps` / `preemption_steps` arrays. Path steps carry `start`/`stop`; Evidence bullets use clickable `range:L0/HI` to zoom and place C1–C2 on the episode
`compare_performance`	optional `tab_a` / `tab_b`	Read-only: structured A vs B metric deltas + confidence (two tabs); `data.regression_type` classifies the primary delta as `execution` / `scheduling` / `synchronization` / `migration` / `load_balance` / `unknown` (legacy `classification` values like `thrashing` / `load_imbalance` / `tick_health` are preserved alongside it)

Tool	Parameters / targets	Effect
`generate_report`	optional `report_type`, `finding_id`	Read-only: typed engineering markdown (`executive` / `performance` / `root_cause` / `regression` / `optimization` / `bug` / `ci`); call `export_report` to save
`check_budget`	optional `budgets`, `tasks`	Read-only: compare per-task WCET/response/deadline metrics against budgets (host builds rows from findings when `tasks` omitted)
`optimize`	optional `limit` (default 5)	Read-only: evidence-backed mitigation ideas (estimate disclaimer)
`regression_explain`	optional `tab_a` / `tab_b`	Read-only: compare two tabs then narrate the primary regression; includes the same `regression_type` classification
`bookmark_finding`	`time`, `kind` (`root_cause` / `evidence` / `correlated` / `reference`); optional `note`	GUI: pin a semantic investigation annotation (Apply required)
`investigation_replay`	optional `finding_id`, `conclusion`, `tools_run`, `evidence_times`	Read-only: structured investigation replay card

Tool	Parameters / targets	Effect
what_if	change; optional task	Read-only: heuristic slice-replay what-if (migrations / blocking / load balance; not FreeRTOS kernel)
optimize_experiment	optional task, limit (1–12, default 5)	Read-only: run ranked automatic pin/priority/contention/migration experiments
analyze_traces	(none)	Read-only: rank all loaded tabs by scheduling behavior
baseline_score	optional task, baseline, snapshot	Read-only: score current per-task metrics (WCET/blocking/migrations/response against the stored historical baseline; flags |z|>2
recommend_experiments	optional finding_id, task, limit (1–20, default 5)	Read-only: suggest simulation / firmware / measurement validation experiments from findings heuristics
export_investigation	optional finding_id, conclusion, tools_run, evidence_times	Download the completed investigation (finding, tools run, queries, evidence, conclusion, confidence, alternatives) as a JSON package

Tool	Parameters / targets	Effect
`detect_priority_inversion`	optional `task`, `window`	Read-only: scan priority-inheritance boost episodes for L/M/H inversion suspects (high/medium/low task, mutex, time, duration)
`find_related_findings`	optional `finding_id`, `task`, `metric`, `window`, `limit` (1–40, default 10)	Read-only: relate Analysis Findings by shared task, metric keyword, evidence-time proximity, or severity adjacency
`compare_tasks`	`task_a`, `task_b`; optional `metrics`	Read-only: side-by-side execution/blocking/migrations/priori inheritance delta table between two tasks
`explain_finding`	optional `finding_id`, `level` (`quick` / `technical` / `deep`)	Read-only: explain one Analysis Finding at the chosen depth (host-side; uses finding text plus hypotheses)
`interpret_query`	`question`	Read-only: turn a free-form question into an explicit investigation mode/scope before other tools run

Tool	Parameters / targets	Effect
validate_experiment	optional expected, actual (metric → signed percent)	Read-only: compare expected experiment deltas with actual A vs B / what-if results (VALIDATED / PARTIALLY VALIDATED / DISPROVED)
manage_hypotheses	hypothesis_id, status (supported / possible / rejected / need_evidence); optional reason, finding_id	Read-only: mark one investigation hypothesis status
plan_investigation	optional question, finding_id	Read-only: rank hypotheses and the cheapest tool sequence
suggest_scope	optional question	Read-only: recommend task / related tasks / time window
detect_contradictions	optional hypothesis, metrics	Read-only: verdict SUPPORTED / CONTRADICTED / INSUFFICIENT
assess_evidence_sufficiency	optional tools_run	Read-only: STOP INVESTIGATION / CONTINUE / REVISE HYPOTHESIS
cluster_findings	(none)	Read-only: group related findings into incidents

Tool	Parameters / targets	Effect
`generate_fingerprint`	(none)	Read-only: HIGH/MEDIUM/LOW scheduling, sync, and timing bands
`find_similar_investigations`	optional `limit`	Read-only: match fingerprint against recorded experiment outcomes
`regression_localize`	optional `label_a`, `label_b`	Read-only: localize A vs B inflation to a task and region
`build_causal_chain`	(none)	Read-only: causal / correlated / temporal edges (never silent causation)
`generate_experiment_plan`	optional `task`, `limit`	Read-only: ranked firmware / what-if experiments
`record_experiment_outcome`	optional `change`, `predicted`, `actual`, `quality`	Read-only: store outcome for later similar-case matching
`score_investigation`	optional `tools_run`, `conclusion`, `confidence`, `elapsed_s`	Read-only: evidence efficiency, cost, false-confidence, falsification, scope, stop
`analyze_temporal_causality`	optional `task`	Read-only: happens-before chain from Findings times

Tool	Parameters / targets	Effect
`build_task_dependency_graph`	optional `task`	Read-only: BTF wait/preempt/migrate/PI graph; 2-hop neighborhood + upstream tasks
`decompose_response_time`	optional `task`	Read-only: relative delay-component shares
`rank_root_causes`	(none)	Read-only: rank causes from findings/hypotheses
`verify_claim`	`claim`; optional `claim_type`, `subject`, `object`, `evidence`	Read-only: SUPPORTED / PARTIAL / UNSUPPORTED
`challenge_conclusion`	optional `conclusion`	Read-only: alternatives and missing evidence
`investigation_memory`	optional `action` (`recall` / `store`), `record`, `limit`	Read-only: persist/recall similar cases
`cluster_incidents`	optional `window_ns`	Read-only: time-proximity incident clusters
`close_investigation`	optional `conclusion`, `confidence`	Read-only: close the case envelope
`analyze_distribution`	optional `values`, `metric` (`auto` / `execution` / `blocking` / `priority_inheritance` / `tick`), `task`	Read-only: p50/p90/p95/p99/p99.9, stddev, CV, 3-sigma outlier rate. Statistics **Query with AI…** on a distribution chart harvests the open plot’s samples.

Tool	Parameters / targets	Effect
`analyze_periodicity`	optional `times`, `expected`, `source` (`auto` / `tick` / `sti` / `isr` / `timer` / `release`), `task`, `durations`	Read-only: expected vs p50/p99/max, RMS and peak-to-peak jitter, kind
`summarize_investigation_context`	optional `conclusion`, `tools_run`	Read-only: compact investigation snapshot

Models without native tool calling can emit a fenced “‘`btftool` JSON block; the viewer renders the same GUI cards. Prefer a tool-capable model for investigation-heavy workflows.

Desktop and web behavior

BTFViewer Desktop and Web are intended to provide the **same AI investigation workflow, tool behavior, evidence model, and validation rules**. For normal use, there is no separate “Desktop workflow” or “Web workflow” to learn.

Use whichever frontend fits your environment:

- **Desktop** — convenient for local files, native save dialogs, and local/private AI endpoints.
- **Web** — convenient when you want a browser-only viewer or a hosted/development deployment.

The differences are mostly platform integration details rather than AI capabilities.

Platform detail	Desktop	Web
AI tools, Investigation Case, Evidence panel, validator	Same behavior	Same behavior
Task/event/region AI actions	Same behavior	Same behavior

Platform detail	Desktop	Web
Model picker and endpoint configuration	Supported	Supported
Reports / investigation export	Native save dialog	Browser download
In-chat diagrams	Rendered in the desktop UI	Rendered as inline browser content
Self-signed HTTPS endpoint	Can optionally allow self-signed TLS per preset	Browser/OS certificate policy still applies
Local `file://` launch	Not applicable	Cross-origin requests may be blocked; use the development/preview server when needed

> **User takeaway:** AI analysis should produce the same investigation and evidence on Desktop and Web. Platform-specific differences matter mainly when configuring endpoints, certificates, downloads, or browser networking.

Detailed platform-specific setup problems are documented in **Troubleshooting** below rather than treated as separate AI features.

Troubleshooting

Symptom	Cause	Try
Authentication	Missing key or sign-in	Settings → AI → Sign in / API key

Symptom	Cause	Try
Self-signed TLS	Private CA or self-signed HTTPS gateway	Desktop: **Allow self-signed TLS**
Model picker	Typed id is not served	Refresh the list and pick a served id
Web: Failed to fetch / CORS	Browser blocked a cross-origin call (`file://` sends `Origin: null`)	Prefer `npm run dev` / `make preview` (both proxy Ollama), or see Opening the web app from `file://`
401 / 403	Missing or rejected key / origin	Settings → AI → Sign in or API key (`OPENAI_API_KEY` / `GEMINI_API_KEY` / `OLLAMA_API_KEY`; local Ollama needs none)
`CERTIFICATE_VERIFY_FAILED` / self-signed TLS	Private CA or self-signed HTTPS gateway	Desktop: Settings → AI → **Allow self-signed TLS**. Web: trust the cert in the OS/browser, use `http://` on a private LAN, or use the Desktop app

Symptom	Cause	Try
Chat probe timed out / The `read operation timed out`	`GET /models` lists ids only; inference is slow or hung	**Test connection** POSTs `/chat/completions` (non-streaming, 120s). Warm the model (`ollama run MODEL`) and retry. Debug with the curl probe below; if curl hangs too, the gateway's chat upstream is stuck. Try `"stream": true` if non-stream never returns. Lower context length on a VRAM-tight local host.
Model not found	Typed id is not served	Refresh the Model list (or Test connection) and pick a served id from the dropdown, or `ollama pull` it
Gemini HTTP 400 `thought_signature`	Gemini 3 requires a thought blob on tool follow-ups	Retry the question — the viewer echoes Gemini thought signatures

Symptom	Cause	Try
Raw “‘`btftool` JSON instead of native tool calls	Model lacks (or skips) function calling	Same cards either way (one object, JSON array, or several objects per fence) — **Apply** or enable **Auto-apply GUI actions**. Switch to `qwen2.5:7b` / `llama3.1:8b` / `gpt-4o` / Gemini for native calls
Ask times out (over 120s) or stays on Waiting…	Cold start, CPU offload, or VRAM spill	**Stop** (composer icon), warm with `ollama run MODEL`, retry. Use **Clear** between long threads. Smaller model or shorter Statistics scope if the Findings card is huge
Later turns ignore earlier facts	Chat history exceeded the context window	**Clear** on the AI bar, or **Analysis → Query with AI…** / toolbar **Compare → Query with AI…** for a fresh scoped prompt
Need raw AI request/response dumps (Desktop)	Debugging tool rounds / provider quirks	Settings → AI → **Log MCP messages to file** (off by default). Appends to `./ai_mcp_messages.log`; delete when finished

Test connection curl

Same body the viewer sends for **Test connection** (replace BASE, MODEL, and KEY):

```
curl -vk --max-time 180 \
  -H "Authorization: Bearer KEY" \
  -H "Content-Type: application/json" \
  -d
   ↪  '{"model":"MODEL","stream":false,"messages":[{"role":"user","content":"Reply
   ↪  with JSON only: {\"ok\":true}"}],"max_tokens":24}' \
  BASE/chat/completions
```

Opening the web app from `file://`

A page opened straight from disk sends `Origin: null`, which Ollama rejects with 403 — the browser then reports only `Failed to fetch`. Serving the app over http avoids this entirely (`npm run dev` / `make preview` proxy Ollama for you), and the Desktop app is not affected at all.

To keep using `file://`, allow every origin on the Ollama side:

```
# Server started from a terminal
OLLAMA_ORIGINS="*" ollama serve

# macOS menu-bar app (Ollama.app) — a shell variable does not reach it
launchctl setenv OLLAMA_ORIGINS "*"   # then quit Ollama and reopen it
```

Verify the change took effect; expect `200` and an `Access-Control-Allow-Origin` header:

```
curl -s -D - -o /dev/null -H "Origin: null" http://localhost:11434/v1/models
 ↪  \
  | grep -iE "^HTTP|access-control-allow-origin"
```

If a `file://` page is still refused, list the null origin explicitly with `OLLAMA_ORIGINS="*,null"`. Note that `*` lets **any** page you visit reach your local models; undo it with `launchctl unsetenv OLLAMA_ORIGINS` when done.

CLI regression gate

Desktop headless CI can compare a candidate trace to a baseline and optionally ask the configured AI for a short narrative:

```
python builds/btf_viewer.py analyze candidate.btf --baseline baseline.btf
↪  --fail-on-regression
python builds/btf_viewer.py analyze candidate.btf --save-baseline
↪  /tmp/base.json
python builds/btf_viewer.py analyze candidate.btf --baseline /tmp/base.json
↪  --fail-on-regression --ai
python builds/btf_viewer.py ai-test --dataset tests/ai --fail-under 70
python builds/btf_viewer.py ai-test --config examples/ai/benchmark.xml -o
↪  AI_BENCHMARK.md
python builds/btf_viewer.py ai-test --config
↪  examples/ai/benchmark-selfsigned.xml --insecure
```

Or `make -C BTFViewer ai-test` (`AI_DATASET`, `AI_FAIL_UNDER`) and `make -C BTFViewer ai-test-live` (`AI_CONFIG`, optional `AI_MODELS` filter, writes AI_BENCHMARK.md). Dataset, scoring rules, and remaining in-app work: Benchmark / evaluation suite.

See also Export → Headless CLI in the user guide.

Benchmark / evaluation suite

Offline `ai-test / runOfflineBenchmark` already ships. Live runs read **model id, base URL, TLS, and API key** from a suite XML (`--config examples/ai/benchmark.xml`) and write AI_BENCHMARK.md. Commands: CLI regression gate.

The capability matrix above is qualitative (small local vs 9B+ vs cloud). The suite turns those expectations into repeatable measurements: **which model is most reliable for BTF Viewer trace investigation**, not which model is largest or "smartest."

Scope

Keep the live set focused on:

- **Gemini cloud models**
- **Local Ollama models that are practical on a typical developer workstation**

Do **not** pick local models only because they are newest or largest. Measure models that can run alongside BTF Viewer, Ollama, and the AI context/tooling workload.

Recommended models

Gemini (configurable; newer ids can be added without changing the runner):

- **Gemini 3.6 Flash** — high-reasoning cloud reference
- **Gemini 3.1 Flash-Lite** — fast/efficient cloud reference

Local — developer workstation:

- **Qwen3.5 9B** (`qwen3.5:9b`) — shipped in-app default; primary practical local investigator
- **Qwen3.5 27B** — higher-quality local / memory-and-latency stress test
- **Qwen3.8 27B** (`qwen3.8:27b`) — newer Qwen 27B local comparison
- **Gemma 4 26B** — non-Qwen local comparison

Older 7B/14B ids stay optional. Do not include 3B-class models — they skip native tool calls and fail the investigation suite.

```
Local AI — developer workstation
│
├── Practical / default
│   └── Qwen3.5 9B
│
├── High-quality local
│   ├── Qwen3.5 27B
│   ├── Qwen3.8 27B
│   └── Gemma 4 26B
```

A 9B model may beat a 27B model on this app if the larger id only slightly improves accuracy while blowing latency and memory. Measure **diagnostic quality** and **practical system performance**.

Do not hard-code the model list into the runner. Copy examples/ai/benchmark.xml (self-signed TLS: benchmark-selfsigned.xml):

```
<ai-benchmark version="1">
  <dataset>tests/ai</dataset>
  <fail-under>0</fail-under>
  <output>AI_BENCHMARK.md</output>
  <endpoint>
    <base-url>http://localhost:11434/v1</base-url>
    <tls-verify>true</tls-verify>
    <timeout-s>360</timeout-s>
  </endpoint>
  <models>
    <model id="qwen3.5:9b"/>
    <model id="qwen3.8:27b"/>
    <model id="gemini-3.6-flash" preset="gemini">
      <base-
↪   url>https://generativelanguage.googleapis.com/v1beta/openai</base-url>
      <api-key env="GEMINI_API_KEY"/>
    </model>
    <model id="gemini-3.1-flash-lite" preset="gemini">
      <base-
↪   url>https://generativelanguage.googleapis.com/v1beta/openai</base-url>
      <api-key env="GEMINI_API_KEY"/>
    </model>
  </models>
</ai-benchmark>
```

```
<!-- Self-signed / private CA gateway -->
<endpoint>
  <base-url>https://llm.internal.example:8443/v1</base-url>
  <tls-verify>false</tls-verify>
  <api-key env="GATEWAY_API_KEY"/>
</endpoint>
```

<api-key env="VAR"> reads the environment first, then any text inside the element. Omit the text (and do not commit secrets). `tls-verify` false, or `ai-test --insecure`, skips certificate checks on Desktop. `--models id1,id2` selects a subset of <model> entries. For Ollama, list the ids you actually have pulled. Record the exact model identifier and runtime configuration.

In-app picker (not shipped): **Settings → AI → Benchmark** with checkboxes for Gemini and local Ollama models, then **Run Benchmark**.

Dataset

`tests/ai/` holds known traces for the major diagnostic scenarios. Each case has **expected facts**, not an exact natural-language answer:

```
tests/ai/
├── migration_thrash.btf
├── mutex_contention.btf
├── priority_inversion.btf
├── deadline_miss.btf
├── load_imbalance.btf
├── trace_regression.btf
├── explain_region.btf
├── adversarial_mutex_vs_starvation.btf
├── adversarial_exec_vs_preemption.btf
├── adversarial_correlation_not_cause.btf
├── adversarial_out_of_scope_time.btf
├── period_jitter.btf
├── waiter_owner_handoff.btf
├── stats_page_next_check.btf
├── response_vs_blocking.btf
├── preempt_matrix_vs_chain.btf
└── mutex_block_vs_wait_queue.btf
```

Adversarial cases (kind `adversarial`) use a decoy finding or timestamp. The obvious answer is wrong:

Case	Decoy	Actual
`adversarial_mutex_vs_starvation`	mutex contention	CPU starvation / preemption
`adversarial_exec_vs_preemption`	long execution / WCET	preemption
`adversarial_correlation_not_cause`	ISR caused Comm latency	correlation, no causal link
`adversarial_out_of_scope_time`	diagnose `jump:9000`	timestamp outside the cursor window
`period_jitter`	tick health / tickless	task inter-arrival; open **Period / Jitter**
`waiter_owner_handoff`	kernel wait-queue	heuristic mutex handoff; open **Waiter × Owner**
`stats_page_next_check`	invent `detect_timeline_anomalies`	open **Timeline Anomalies** / **Worst Events**
`response_vs_blocking`	Blocking Time is end-to-end response	open **Response Time**
`preempt_matrix_vs_chain`	invent `detect_preemption_matrix`	open **Preemption Matrix**
`mutex_block_vs_wait_queue`	reconstruct the kernel wait queue	open **Mutex Blocking**

```yaml
id: migration_thrash
trace: migration_thrash.btf
expected:
  finding_types: [migration, load_balance]
  tasks: [CS[22]]
  evidence:
    required_metrics: [migrations]
  allowed_tools: [detect_anomalies, correlate_events, query_raw_metric]
  forbidden:
    invented_task_names: true
    out_of_scope_timestamps: true
```

That keeps scoring robust against harmless wording differences.

Evaluation metrics

Metric	What it measures
Finding identification	Did the model identify the expected problem?
Evidence accuracy	Are cited metrics/events actually present? `required_metrics` also accepts Statistics page titles (Period / Jitter, Waiter × Owner, Timeline Anomalies, ...) and common aliases (`Period/Jitter`, `Waiter x Owner`)
Timestamp validity	Are `jump:TIME` values real and in scope?
Task-name validity	Did the model use only known task names?
Tool selection	Did it call appropriate investigation tools?
Tool-chain quality	Did it gather enough evidence before concluding?
Root-cause accuracy	Does the conclusion match the expected diagnosis?
Alternative handling	Did it consider plausible alternatives?
Confidence calibration	Is confidence consistent with the available evidence?
Response completeness	Did it answer the investigation question completely?
Latency	How long did the investigation take?
Tool-call count	How many tool rounds were required?
Peak memory	How much RAM was consumed during inference?
Time to first token (TTFT)	How quickly did the model begin responding?
Generation throughput	Sustained tokens/sec during the investigation
Investigation success rate	Percentage of cases completed correctly within the configured time/resource limit
False-causal rate	Claimed a causal link the case marks as coincidence / non-causal (0–100, higher is worse)
False-confirmation rate	Confirmed the decoy finding (`trap_phrases`) instead of the real cause

Metric	What it measures
Unsupported-claim rate	Share of validator claims that fail task/time/scope checks
Premature-conclusion rate	High confidence or a conclusion before required tools ran

For local runs, memory and latency are first-class. A slightly more accurate model that is unusable under memory pressure should not automatically rank higher.

Level 1 — tool / evidence correctness: valid tool, parameters, task, timestamp, and scope. Isolates tool-use bugs from reasoning quality.

Level 2 — diagnostic correctness: expected vs actual diagnosis, evidence, and alternatives. A convincing explanation is not enough.

Headline score: weighted engineering score (Finding / Evidence / Tool use / Root cause / Calibration / Safety), not a probability of correctness. Keep the component scores visible. PASS is overall ≥ 70.

Safeguards the suite must test: no invented task names, metric values, or `jump:TIME`; no timestamps outside the cursor region; no unsupported conclusions presented as confirmed; evidence must match tool results; heuristic what-if stays labeled as estimates; the model must not claim a simulation is a measured result.

Model matrix

Same suite against selected Gemini and local Ollama models. Recorded 2026-08-14; full case tables: AI_BENCHMARK.md.

Model	Category	Finding	Evidence	Root cause	Calibration	Notes
Gemini 3.1 Flash-Lite	Cloud / fast	**79**	64	**71**	80	overall **81**, 5.5s; tool follow-up

Model	Category	Finding	Evidence	Root cause	Calibration	Notes
Gemini 3.6 Flash	Cloud	—	—	—	—	overall **75**, 6/7; free-tier 429 split the part dump
Qwen3.5 9B	Local / practical	**79**	**86**	57	80	overall **82**, 10.7s
Qwen3.5 27B	Local / high-quality	71	71	71	80	overall **80**, 64.4s
Gemma 4 26B	Local / high-quality	71	57	**86**	80	overall **80**, 72.6s; 5/7 PASS

Live `--config` runs a tool-result follow-up when the first turn is tools-only (or planning text without a Confidence line). Single-turn scores are not comparable.

Context-size (Findings + tools + history). Do not judge a local model only by tokens/sec — check tool use and grounding as context grows:

Context	Purpose
8K	Minimum investigation workload
16K	Typical investigation
32K	Large Findings / multi-tool investigation
64K	Stress test, if supported

Practical comparison on a developer workstation:

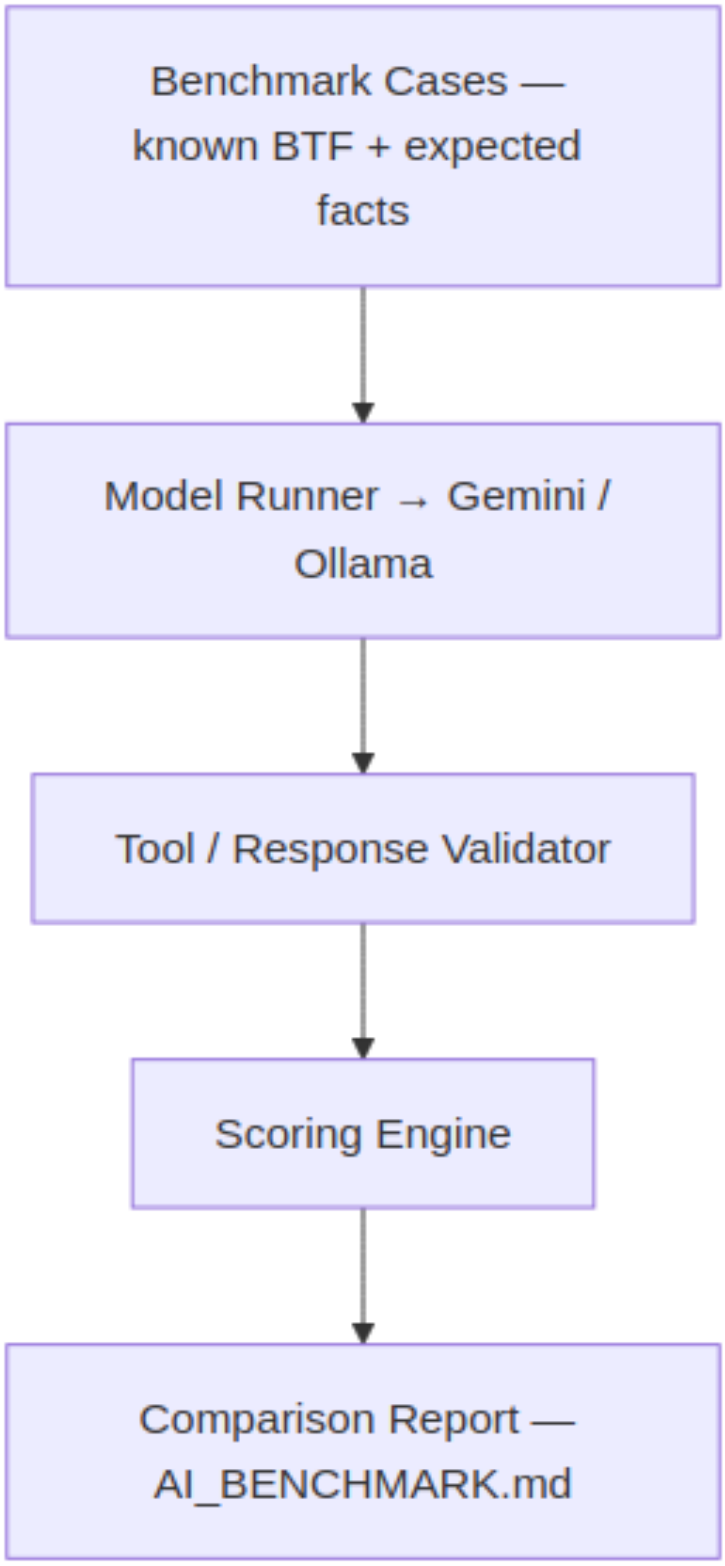
```
Gemini 3.6 Flash / Gemini 3.1 Flash-Lite
      vs
Qwen3.5 9B        (shipped default)
Qwen3.5 27B
Qwen3.8 27B
Gemma 4 26B
```

Does extra local capacity improve investigation quality enough to justify memory and latency?

Reproducibility and architecture

Each run should save timestamp, app version, dataset version, model ids, endpoint config, cases, prompts, tool calls/results, final responses, scores, and timing. A run ID (`AI Benchmark #2026-08-13-001`) keeps results comparable when model behavior drifts without a viewer code change.

`--fail-under N` can fail CI when a model drops below a threshold (live often wants `0` so HTTP errors still write the report).


Benchmark Cases — known BTF + expected facts
Model Runner → Gemini / Ollama
Tool / Response Validator
Scoring Engine
Comparison Report — AI_BENCHMARK.md

Investigation Case

Desktop and Web share one **Investigation Case** model (`btf-investigation-case`): question, scope (trace / C1–Cn / tasks / cores), hypotheses with status (**supported** / **possible** / **need evidence** / **rejected**), evidence graph, coverage, falsification checks, conclusion, and validation. Engines: `btf_viewer_pkg/ai_case.py` ↔ `web/src/utils/aiCase.js` (see Implementation notes).

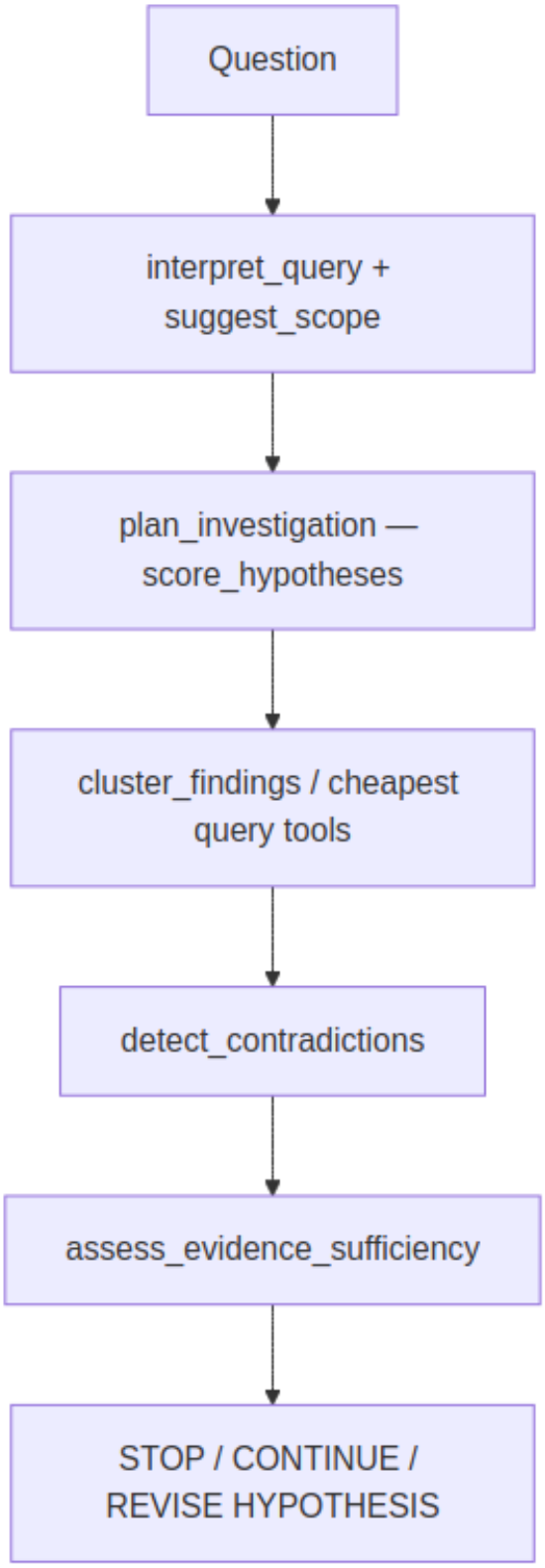
After each final assistant reply a host-side **validator** extracts `jump:TIME` and `Task[id]` claims and flags invented names or timestamps outside the cursor window. **Test connection** appends a **model capability** card (live chat / structured output / tool calling overlay on a 3B vs 7B+ heuristic). Headless eval:

```
make -C BTFViewer ai-test
# or: python builds/btf_viewer.py ai-test --dataset tests/ai --fail-under 70
```

Modes (Quick / Diagnose / Compare / Optimize / Report) map onto existing templates; they do not add new tools. Always confirm on the timeline.

Investigation planner

Host-side planner (`btf_viewer_pkg/ai_planner.py` ↔ `web/src/utils/aiPlanner.js`). **Cheapest evidence first.** User-facing loop: README → Investigation planner.

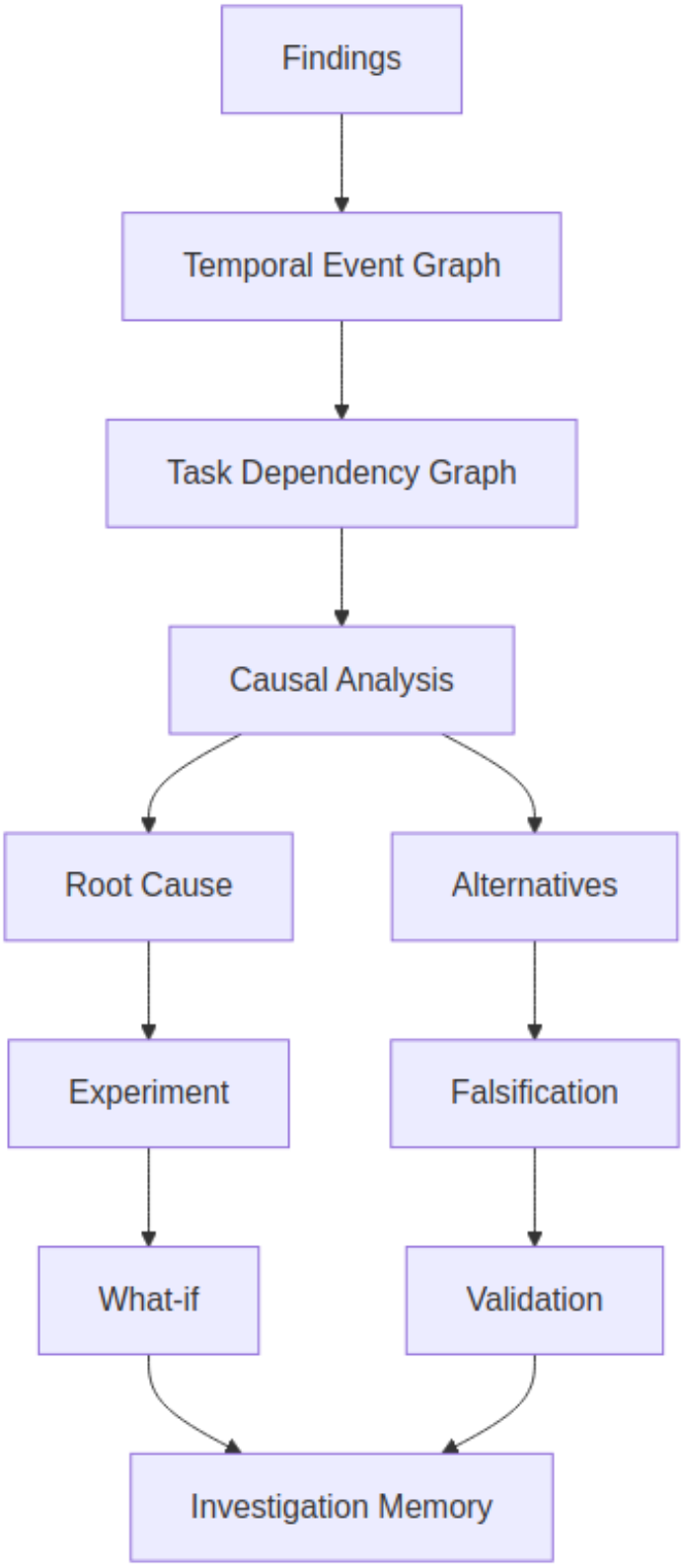

Question
interpret_query + suggest_scope
plan_investigation — score_hypotheses
cluster_findings / cheapest query tools
detect_contradictions
assess_evidence_sufficiency
STOP / CONTINUE / REVISE HYPOTHESIS

Tool / helper	Host behaviour
`plan_investigation`	Rank hypotheses and a cheap tool sequence from findings + question
`suggest_scope`	Task, related tasks, evidence times (or current cursors)
`detect_contradictions`	`SUPPORTED` / `CONTRADICTED` / `INSUFFICIENT` (e.g. execution ≫ blocking vs mutex hypothesis)
`assess_evidence_sufficiency`	Coverage heuristic → stop / continue / revise
`score_hypotheses`	Evidence-weighted scores (not a GUI tool)
`cluster_findings`	Group by shared task or pattern
`generate_fingerprint`	HIGH / MEDIUM / LOW scheduling, sync, timing bands
`find_similar_investigations`	Jaccard-style match vs recorded experiment outcomes
`regression_localize`	A vs B deltas → task / region / likely mechanism
`build_causal_chain`	Edges tagged causal / correlated / temporal; disclaimer required
`generate_experiment_plan`	Ranked pin / contention / priority experiments
`record_experiment_outcome`	Persist outcome (Desktop `[ai] experiment_outcomes`, Web `localStorage`)
`score_investigation`	Phase 3 extras: `evidence_efficiency`, `investigation_cost`, `false_confidence`, `falsification_quality`, `scope_accuracy`, `stop_efficiency` (also spread into `score_benchmark_case`, with adversarial rates)

Do **not** add chat templates after `auto_investigate`.

Causal and temporal engines

Host-side heuristics (`btf_viewer_pkg/ai_causal.py` ↔ `web/src/utils/aiCausal.js`) over Analysis Findings — not a FreeRTOS scheduler replay. User-facing loop stays on Investigation planner. Diagnose / Investigate / Auto investigate walk explanation tools before experiments: `graph` → `temporal` → `rank_root_causes` → `challenge_conclusion`, then `what_if`.


Findings
Temporal Event Graph
Task Dependency Graph
Causal Analysis
Root Cause
Alternatives
Experiment
Falsification
What-if
Validation
Investigation Memory

Tool / helper	Host behaviour
`analyze_temporal_causality`	Happens-before chain from finding times (`jump:TIME`)
`build_task_dependency_graph`	BTF sync/preempt/migrate/PI graph (finding-wording fallback); optional `task` neighborhood
`decompose_response_time`	Relative delay shares (mutex, preemption, migration, execution, scheduler)
`rank_root_causes`	Rank hypotheses or finding buckets
`verify_claim`	`SUPPORTED` / `PARTIAL` / `UNSUPPORTED` vs findings and cursors
`challenge_conclusion`	Alternatives and missing evidence
`investigation_memory`	Store/recall (Desktop `[ai] investigation_memory`, Web `localStorage`)
`cluster_incidents`	Group findings by time proximity
`close_investigation`	Record conclusion and close the case
`analyze_distribution`	p50 / p90 / p95 / p99 / p99.9, stddev, CV, 3-sigma outlier rate; BTF execution/blocking/PI/tick harvest
`analyze_periodicity`	Period/jitter from tick, STI, ISR, timer, or task-release times; kind = drift vs jitter vs WCET vs scheduler
`summarize_investigation_context`	Compact findings, hypotheses, and tools run

Engine limits

Engine	What it is	What it is not
`analyze_temporal_causality`	Happens-before from finding `jump:TIME`	Kernel event replay
`build_task_dependency_graph`	BTF sync / preempt / migrate / PI edges; 2-hop `task` neighborhood	Full ISR / object graph
`decompose_response_time`	Relative shares from finding magnitudes	Cycle-accurate milliseconds
`rank_root_causes`	Hypothesis or finding-bucket rank	A probability
`investigation_memory`	Local store / recall notepad	Team knowledge base
`cluster_incidents`	Time-proximity groups	Shared-mutex / causal clustering
`close_investigation`	Case status `closed` plus conclusion	Full firmware A/B lifecycle
`analyze_distribution`	BTF execution / blocking / PI / tick samples (cap 8000)	A response-time series the parser does not have
`analyze_periodicity`	Inter-arrival jitter and kind	A kernel period timer
`simulate_schedule`	LEVEL 1 helper inside `what_if`	A GUI tool or FreeRTOS kernel

Out of scope (do **not** add chat templates for these): trace-to-code (ELF / DWARF), real scheduler or hardware-aware simulation, model routing, automatic benchmark-case generation, natural-language → metric compiler, anomaly discovery without Analysis Findings, shared team investigation database.

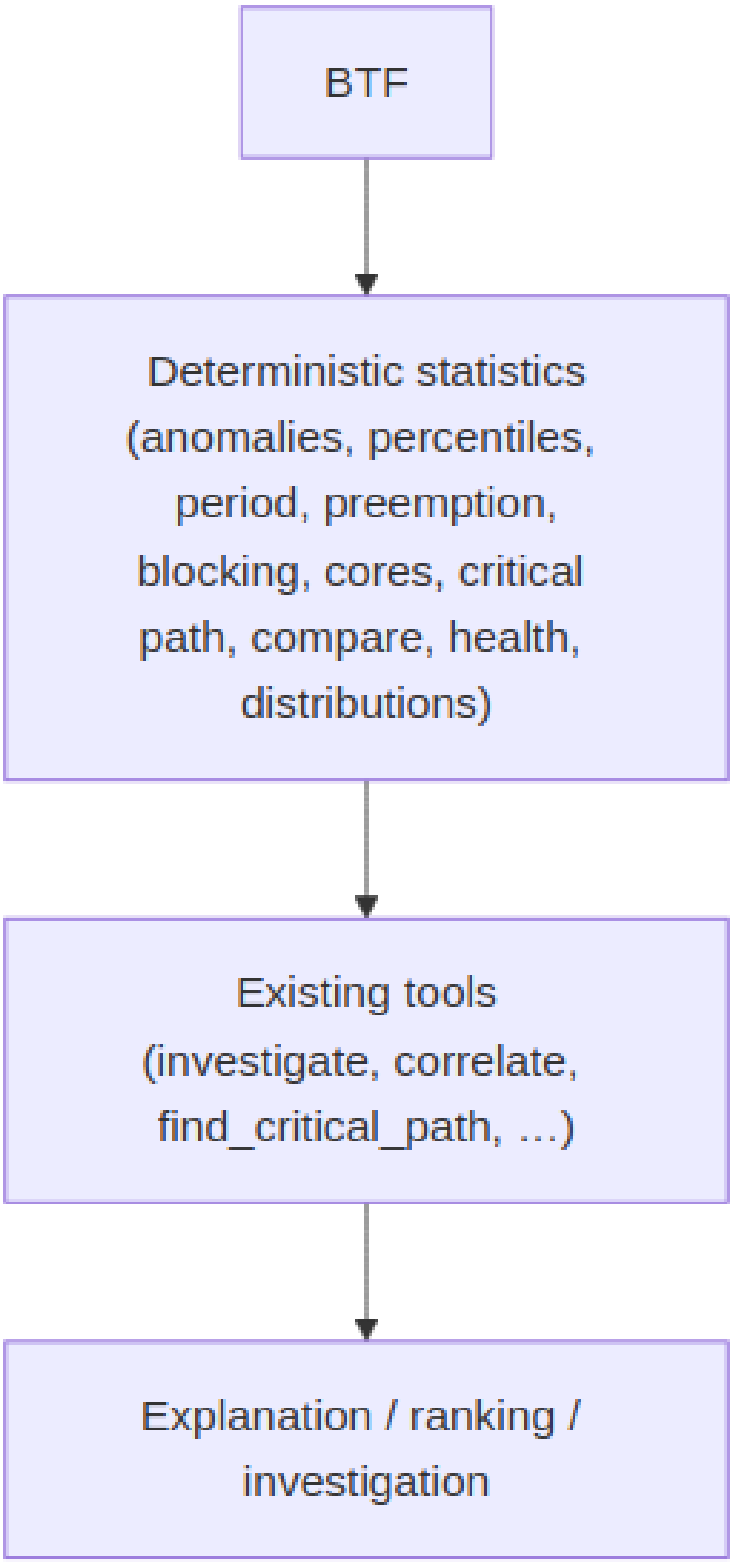
Do **not** add chat templates after `auto_investigate`. Next gains come from deeper engines, not more buttons.

Implementation notes

Tech notes for keeping Desktop and Web in lockstep. User-facing Case / Evidence behaviour: README → Investigation Case. Live suite XML: Benchmark / evaluation suite. Recorded scores: `[AI_BENCHMARK.md](AI_BENCHMARK.md)`.

Analysis vs AI tools

Facts come from BTF Statistics pages first. AI ranks, explains, and navigates those facts. **Do not add AI tools** for work those pages already do (no `detect_timeline_anomalies`, extra jitter tools, or histogram tools). **Do not** invent kernel response time, inspect ELF/source, or simulate the scheduler.


BTF
Deterministic statistics (anomalies, percentiles, period, preemption, blocking, cores, critical path, compare, health, distributions)
Existing tools (investigate, correlate, find_critical_path, ...)
Explanation / ranking / investigation

The shipped loop stays **Triage → Investigate → Verify → Correlate → Critical Path → Dependency Graph → Temporal Causality → Rank → Challenge → What-if → Report**. Improve access to Statistics evidence; do not grow the tool list. User-facing page map: README → BTF analysis pages.

Shared Case / Evidence engines

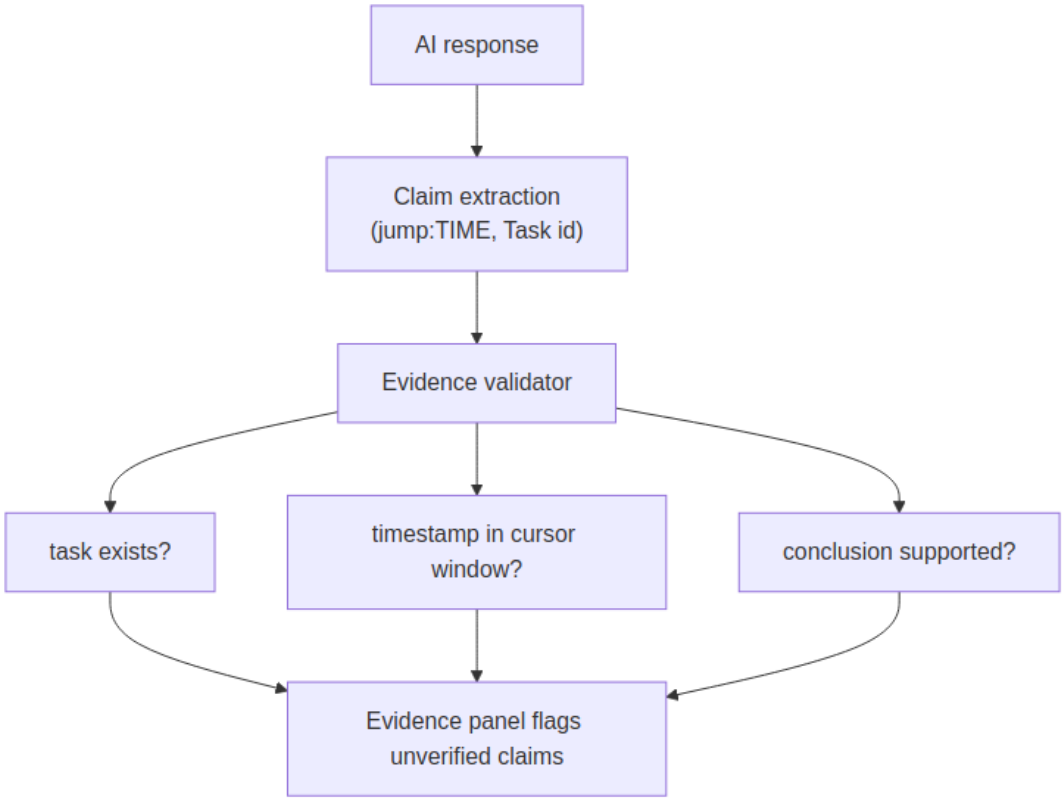
Desktop	Web
`btf_viewer_pkg/ai_case.py`	`web/src/utils/aiCase.js`
`btf_viewer_pkg/ai_investigation.py`	`web/src/utils/aiInvestigation.js`
`btf_viewer_pkg/ai_planner.py`	`web/src/utils/aiPlanner.js`
`btf_viewer_pkg/ai_causal.py`	`web/src/utils/aiCausal.js`
`btf_viewer_pkg/ai_tools.py`	`web/src/utils/aiTools.js`
`btf_viewer_pkg/ai_mermaid.py`	`web/src/utils/aiMermaid.js`

Parity is gated by `tests/test_ai_web_parity.py` (including planner and causal tool names vs Desktop/Web) and `web/tests/aiCase.test.js`. Bundle order: `ai_case` then `ai_investigation` then `ai_planner` then `ai_causal` then `ai_tools`. New `EVIDENCE_PANEL_LABELS` keys must exist in all 8 languages. After AI UI changes: `make -C BTFViewer bundle` and `make -C BTFViewer web`.

UI lockstep: mode chips wrap (`_FlowLayout` ↔ `flex-wrap`). Primary templates are two rows: Analysis Findings / Explain region / Investigate, then **Auto investigate** + **More templates…** (Desktop `break_before` ↔ web `.ai-tpl-row`). Chip min-height 28px, disabled chips/menu items #8a96a8. Findings **Investigate…** uses the same outline style as the other Analysis footer buttons (not accent/primary). **More** templates use the same groups in a 2-column overlay (Desktop `QFrame` ↔ Web body overlay). Trace Compare opens from toolbar **Compare**, not the Statistics footer.

The Desktop `ai-test` CLI and Web `runOfflineBenchmark` share `tests/ai` fixtures (tracked `.btf` stubs + `dataset.json`).

Validator



`validate_ai_response` / `validateAiResponse` runs on the host after the final reply. Prompting still forbids inventing numbers, task names, and `jump:TIME`; the validator is the guard, not the prompt.

Experiment close-out

`validate_experiment` compares expected vs actual signed percents (`VALIDATED` / `PARTIALLY VALIDATED` / `DISPROVED`), updates open hypotheses, and offers **Save to knowledge** (`btfexp:save`). Empty `actual` is filled from the last Trace Compare refresh (including **Scope to cursors**) or `compare_performance` via `experiment_percents_from_compare`. Toolbar **Compare → Validate experiment…** closes the dialog and asks the model to call the tool with actuals omitted. Firmware-change capture remains a user step (new trace + toolbar **Compare**).

Capability, cost, privacy, knowledge

Feature	Host behaviour
Capability probe	**Test connection** lists models, chats with a JSON structured-output probe, then tool-calling (`btf_ping` then `btf_pong`). Live results overlay chat / structured output / tool calling / multi-tool chaining; long context and reasoning stay heuristic.
Cost	A dedicated usage bar shows accumulated `tok · tools · s` (and estimated USD when priced). Evidence uses the full `format_cost_meter` line. **Clear** resets replies, the meter, and current investigation issues.
Privacy	Chip • Local / • Cloud / • Sensitive. Cloud send is blocked when sensitive; otherwise annotations are sanitized and optional task-name aliases apply (`apply_cloud_privacy`).
Knowledge	`investigate` matches user-saved entries (More → **Save current finding…**), then baseline, then the builtin catalog. Typical vs current rates show when both exist.
Interpret	Free-form Ask host-interprets first (`interpret_query`). Templates / modes / **Run investigation** skip confirm.
Tool Why?	Evidence **Investigation** lists each tool with a host-side reason (`btftool:why/name`).

Diagrams

Replies may include ```` ```mermaid ```` **sequence** diagrams (mutex take/give, block/resume, priority boost / L/M/H) and `graph LR` / **flowchart** core-migration graphs (counts on edges). Pipe **Markdown tables** (and a sanitized HTML `<table>` copied from Findings) render as HTML tables in the reply pane. In-chat Markdown / HTML tables use the same sanitizer. The Evidence panel adds its own auto-generated `graph`

`TD` **Investigation tree** (root-cause chain steps as boxes, alternative hypotheses as rounded nodes branching off the finding) whenever `investigate` returns a chain — same renderer, same click/zoom rules below. Sequence, flowchart, and investigation-tree palettes follow **dark and light** theme (`mermaid_palette` / `mermaidPalette`); **Save As…** HTML conversation export uses the **light** palette for white paper.

- Click a **task** node to lock-highlight that timeline row (`Low[266] (Core 0)` resolves to `Low[266]`).
- Click a **core** node (`Core_0`, `C0`, `C1`) to switch to Core View and scroll to that core.
- Mutex hex and other unresolved labels do nothing (the timeline stays undimmed).
- Click empty figure area to open a larger zoom window (scroll to zoom 0.5–6×; **Esc** or **Close**). Trackpad pinch is treated as scroll.
- The link row under the figure has the same targets.
- **Save As…** HTML keeps inline SVG with clickable nodes (chat zoom wrappers are omitted).

Documentation navigation

Document	Question answered
README.md	How do I use BTFViewer?
WORKFLOWS.md	How do I diagnose a problem?
STATISTICS.md	What does this measurement mean?
AI.md	How does AI-assisted investigation work?